Managing Scheduling

Kubernetes Production · Lesson 12

Built from the official Kubernetes documentation

`kubernetes.io/docs`

Source: kubernetes.io/docs · Apache 2.0

Learning Objectives

By the end of this lesson you will be able to:

- Explain how the **scheduler** picks a node (filtering + scoring)
- Constrain Pods with **nodeSelector**, labels and **nodeName**
- Use **node affinity** and **inter-pod affinity / anti-affinity**
- Apply **taints & tolerations** to repel or admit Pods
- Set **resource requests & limits** and read **QoS classes**
- Spread Pods evenly with **topology spread constraints**

> Reference: *Scheduling, Preemption and Eviction* — kubernetes.io/docs/concepts/scheduling-eviction

Source: kubernetes.io/docs · Apache 2.0

12.1 How the Scheduler Works

Source: kubernetes.io/docs · Apache 2.0

How the Scheduler Picks a Node

`kube-scheduler` watches for Pods with **no assigned node** and binds each to a suitable one in **two steps**:

Step	What it does
Filtering	Keeps only nodes where it is *feasible* to run the Pod (resources, taints, affinity) — the *feasible* set
Scoring	Ranks the feasible nodes and picks the **highest score** (ties broken at random)

- If the feasible set is **empty**, the Pod stays **Pending**.
- The chosen node is **bound** to the Pod; its kubelet then starts the containers.

> CKA tip: `kubectl describe pod` → the **Events** section tells you *why* a Pod is unschedulable (e.g. `0/3 nodes are available: insufficient cpu`).

Ways to Constrain a Pod to Nodes

Kubernetes offers several mechanisms, from simplest to most expressive:

- **nodeSelector** — match a Pod to nodes that carry **all** given labels
- **Node affinity / anti-affinity** — richer rules with operators and weights
- **Inter-pod affinity / anti-affinity** — place Pods relative to **other Pods**
- **nodeName** — bypass the scheduler and pin to one named node
- **Topology spread constraints** — distribute Pods across failure domains

> All label-based methods rely on **node labels** — many are populated automatically (e.g. `topology.kubernetes.io/zone`, `kubernetes.io/hostname`).

Source: kubernetes.io/docs · Apache 2.0

12.2 nodeSelector & nodeName

Source: kubernetes.io/docs · Apache 2.0

Node Labels & nodeSelector

`nodeSelector` is the **simplest** form of node selection: the Pod runs only on nodes that carry **all** the listed labels.

```
# Label a node so Pods can target it
kubectl label nodes node1 disktype=ssd
kubectl get nodes --show-labels
```

```
spec:
  nodeSelector:
    disktype: ssd
```

> Labels with the `node-restriction.kubernetes.io/` prefix **cannot** be set by the kubelet — use them (with the `NodeRestriction` admission plugin) for secure node isolation.

nodeName — Direct Assignment

`nodeName` is the most direct selection method — it **bypasses the scheduler** and the kubelet on that node runs the Pod.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  nodeName: node1
  containers:
  - name: nginx
    image: nginx
```

Limitations — the Pod **fails** if:

- the named node **does not exist** or is **unreachable**
- the node **lacks the resources** to run the Pod
- node names are not stable in cloud autoscaling environments

> Prefer affinity/nodeSelector over `nodeName` for anything but quick tests.

Source: kubernetes.io/docs · Apache 2.0

12.3 Node Affinity

Source: kubernetes.io/docs · Apache 2.0

Node Affinity — Two Rule Types

Node affinity is like `nodeSelector` but far more expressive — and it adds **soft** rules. Two flavours:

Rule	Meaning
`requiredDuringSchedulingIgnoredDuringExecution`	**Hard** — Pod is *not* scheduled unless the rule is met
`preferredDuringSchedulingIgnoredDuringExecution`	**Soft** — scheduler *prefers* matching nodes, schedules anyway if none

- **IgnoredDuringExecution**: if labels change *after* scheduling, the running Pod is **not** evicted.
- If both `nodeSelector` **and** `nodeAffinity` are set, **both** must match.

Source: kubernetes.io/docs · Apache 2.0

Node Affinity — Example

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: with-node-affinity
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: topology.kubernetes.io/zone
            operator: In
            values:
            - antarctica-east1
            - antarctica-west1
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 1
        preference:
          matchExpressions:
          - key: another-node-label-key
            operator: In
            values:
            - another-node-label-value
```

Source: kubernetes.io/docs · Apache 2.0

Operators & Matching Logic

Operator	Matches when the node label…
`In`	value is in the given list
`NotIn`	value is **not** in the list *(anti-affinity)*
`Exists`	key exists (no `values` needed)
`DoesNotExist`	key is absent *(anti-affinity)*
`Gt` / `Lt`	value is greater / less than (**integer** comparison)

- Multiple `nodeSelectorTerms` are **OR-ed** (any term may match).
- Multiple `matchExpressions` within a term are **AND-ed** (all must match).
- Soft rules carry a **`weight` (1–100)**; the scheduler sums the weights of all satisfied preferences and favours the highest-scoring node.

Source: kubernetes.io/docs · Apache 2.0

12.4 Inter-Pod Affinity & Anti-Affinity

Source: kubernetes.io/docs · Apache 2.0

Inter-Pod Affinity & Anti-Affinity

These rules place a Pod **relative to other Pods** already running, based on their **labels** — not node labels.

- **podAffinity** — co-locate Pods (e.g. app next to its cache for low latency)
- **podAntiAffinity** — spread Pods apart (e.g. one replica per node for HA)
- Same hard/soft variants: `requiredDuringScheduling…` and `preferred…`
- The **`topologyKey`** defines the domain that "together" / "apart" means: `kubernetes.io/hostname` = per node, `topology.kubernetes.io/zone` = per zone

> Inter-pod affinity examines labels across many Pods and can **slow scheduling** on large clusters — not recommended beyond a few hundred nodes.

Pod Anti-Affinity — Example

```yaml
spec:
  affinity:
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
          - key: app
            operator: In
            values:
            - web-store
        topologyKey: kubernetes.io/hostname
  containers:
  - name: web-app
    image: nginx
```

- The `labelSelector` picks the Pods to avoid (here `app=web-store`).
- `topologyKey: kubernetes.io/hostname` → at most **one** such Pod per node.

Source: kubernetes.io/docs · Apache 2.0

12.5 Taints & Tolerations

Source: kubernetes.io/docs · Apache 2.0

Taints & Tolerations

Taints let a node **repel** Pods; **tolerations** let a Pod be admitted despite a taint. They are the **opposite** of affinity.

```
# Add a taint
kubectl taint nodes node1 key1=value1:NoSchedule
# Remove it (trailing minus)
kubectl taint nodes node1 key1=value1:NoSchedule-
```

Effect	Behaviour for non-tolerating Pods
`NoSchedule`	Not scheduled here; running Pods stay
`PreferNoSchedule`	Scheduler **tries** to avoid the node (soft)
`NoExecute`	Evicted if running, and not scheduled

Source: kubernetes.io/docs · Apache 2.0

Tolerations in the Pod Spec

A toleration matches a taint when **key** and **effect** match and the operator is `Equal` (values equal) or `Exists` (any value).

```
tolerations:
- key: "key1"
  operator: "Equal"
  value: "value1"
  effect: "NoSchedule"
- key: "key1"
  operator: "Exists"
  effect: "NoExecute"
  tolerationSeconds: 3600
```

- `tolerationSeconds` (NoExecute only): stay bound this long, then evict.
- Empty `key` + `Exists` tolerates **everything**; empty `effect` matches all effects for that key.

The Control-Plane Taint

By default `kubeadm` taints control-plane nodes so **user workloads stay off**:

```
node-role.kubernetes.io/control-plane:NoSchedule
```

- To run a Pod there, add a matching toleration — system add-ons (CoreDNS, kube-proxy) already do.
- On a single-node lab, **remove** the taint to schedule normal Pods:

```
kubectl taint nodes <node> \
  node-role.kubernetes.io/control-plane:NoSchedule-
```

> Kubernetes also adds **node-condition taints** automatically, e.g. `node.kubernetes.io/not-ready` and `node.kubernetes.io/unreachable` (NoExecute).

Source: kubernetes.io/docs · Apache 2.0

12.6 Resources & QoS

Resource Requests & Limits

Each container can declare `requests` (what it needs) and `limits` (its ceiling).

```yaml
spec:
  containers:
  - name: app
    image: nginx
    resources:
      requests:
        cpu: "250m"         # 0.25 core — used for scheduling
        memory: "256Mi"
      limits:
        cpu: "500m"         # throttled above this
        memory: "512Mi"     # OOM-killed if exceeded
```

- **CPU**: cores; `1` = one core, `100m` = 0.1 core (`m` = millicpu).
- **Memory**: bytes; binary suffixes `Ki, Mi, Gi…`, decimal `k, M, G…`.

Source: kubernetes.io/docs · Apache 2.0

How Requests Drive Scheduling

- The scheduler places a Pod only on a node where the **sum of requests** of all Pods still **fits the node's allocatable** capacity — it looks at *requests*, not actual live usage.
- A container **may** use more than its request (if the node has spare), but **never** more than its limit.

At runtime the kubelet enforces limits:

- **CPU over limit** → throttled (slowed, not killed).
- **Memory over limit** → container is **OOM-killed**.

> CKA tip: a Pod stuck `Pending` with `Insufficient cpu/memory` means **no node has enough unreserved requests** — lower the request or add capacity.

Quality of Service (QoS) Classes

Kubernetes derives a QoS class from a Pod's requests/limits; it drives the **eviction order** under node memory pressure.

QoS class	Condition	Evicted
Guaranteed	Every container has CPU & memory **requests = limits** (both set, > 0)	**Last**
Burstable	Not Guaranteed, but **at least one** request or limit is set	Second
BestEffort	**No** requests or limits set on any container	**First**

- View it with `kubectl get pod <name> -o jsonpath='{.status.qosClass}'`.
- Only Pods **exceeding their requests** are prime eviction candidates.

> CKA tip: to guarantee a critical Pod, set `requests == limits` for **every** container, for **both** CPU and memory.

12.7 Topology Spread Constraints

Source: kubernetes.io/docs · Apache 2.0

Topology Spread Constraints

Spread matching Pods evenly across **failure domains** (zones, nodes) for HA and balanced utilisation.

- **maxSkew** — max allowed difference in Pod count between domains (must be > 0)
- **topologyKey** — node label defining a domain (e.g. `…/zone`, `…/hostname`)
- **whenUnsatisfiable** — `DoNotSchedule` (hard) or `ScheduleAnyway` (soft, best-effort)
- **labelSelector** — which Pods are counted toward the skew

> **Skew** = (Pods in a domain) − (global minimum across eligible domains).
> The scheduler keeps it ≤ `maxSkew`.

Source: kubernetes.io/docs · Apache 2.0

Topology Spread — Example

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: example-pod
spec:
  topologySpreadConstraints:
  - maxSkew: 1
    topologyKey: topology.kubernetes.io/zone
    whenUnsatisfiable: DoNotSchedule
    labelSelector:
      matchLabels:
        app: web
  containers:
  - name: web
    image: nginx
```

- Keeps `app=web` Pods within **1** of each other across zones.
- `DoNotSchedule` → leaves the Pod **Pending** rather than break the spread.

Source: kubernetes.io/docs · Apache 2.0

Key Takeaways

- The scheduler **filters** then **scores** nodes; unschedulable Pods stay Pending
- **nodeSelector** = simple label match; **nodeName** bypasses the scheduler
- **Node affinity** adds operators (`In/NotIn/Exists…`) and soft `weight` rules
- **Inter-pod (anti-)affinity** places Pods via labels + a **topologyKey**
- **Taints** repel Pods; **tolerations** admit them (`NoSchedule/PreferNoSchedule/NoExecute`)
- **Requests** drive scheduling; **limits** cap usage; QoS = **Guaranteed/Burstable/BestEffort**
- **Topology spread** balances Pods across domains via `maxSkew` + `whenUnsatisfiable`

Source: kubernetes.io/docs · Apache 2.0

End of Lesson 12

Next: Lesson 13 — Networking & NetworkPolicy

`kubectl taint` · `kubectl label nodes` · `kubectl get pod -o jsonpath='{.status.qosClass}'`

</content>

Source: kubernetes.io/docs · Apache 2.0